Software Requirements Specification

for

Hostel & Mess Management System (HMMS)

Version 1.0 approved

Prepared by Giridhar A (Roll No. 084)

Coimbatore Institute of Technology

September 2026

Table of Contents

Table of Contents ..... **ii**
Revision History ..... **ii**
1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 2
1.4 Product Scope 2
1.5 References 3
2. Overall Description **3**
2.1 Product Perspective 3
2.2 Product Functions 4
2.3 User Classes and Characteristics 4
2.4 Operating Environment 5
2.5 Design and Implementation Constraints 6
2.6 User Documentation 6
2.7 Assumptions and Dependencies 6
3. External Interface Requirements **7**
3.1 User Interfaces 7
3.2 Hardware Interfaces 7
3.3 Software Interfaces 8
3.4 Communications Interfaces 8
4. System Features **8**
4.1 User Authentication and Access Control 8
4.2 Student Resident Onboarding and Profiling 9
4.3 Room Inventory and Occupancy Allocation 10
4.4 Multi-Session Daily Attendance Tracking 10
4.5 Digital Leave Application and Warden Approval 11
4.6 Fee Ledger, Billing Reports, and Arrears Audit 12
4.7 Facility Maintenance and Grievance Ticketing 13
4.8 Mess Dining Menu and Catering Operations 13
4.9 Supervisory Staff Provisioning & Governance 14
5. Other Nonfunctional Requirements **14**
5.1 Performance Requirements 14
5.2 Safety Requirements 14
5.3 Security Requirements 14
5.4 Software Quality Attributes 15
5.5 Business Rules 15
6. Other Requirements **16**
Appendix : Analysis Of Models **19**
Demonstration **30**

Revision History

Name	Date	Reason For Changes	Version
Engineering Design Team	09/10/2026	Initial System Inception & Requirements Elicitation	0.1
Architecture Review Board	09/15/2026	Refinement of Relational Schemas & RBAC	0.5
Full-Stack Implementation Lead	09/22/2026	Alignment with Production React/Node.js/MySQL	0.9
System Architecture Board	09/29/2026	Final Specification Review & Release Approval	1.0 Approved

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) document is to provide a complete, rigorous, and unambiguous description of the software requirements for the Hostel & Mess Management System (HMMS), Version 1.0. This specification outlines the functional capabilities, performance criteria, user interactions, external interfaces, database architectures, and design constraints that govern the design, implementation, and deployment of HMMS.

This document encompasses the entire full-stack software system, including the clientside Single Page Application (SPA), the serverside Representational State Transfer (REST) Application Programming Interface (API), the relational database layer, and all supporting authentication and operational subsystems. It serves as the primary contractual and technical baseline for software developers, system architects, quality assurance engineers, institutional stakeholders, and project supervisors.

1.2 Document Conventions

This document uses the following conventions

Term / Acronym	Definition / Standard Meaning
HMMS	Hostel & Mess Management System
IEEE	Institute of Electrical and Electronics Engineers
SRS	Software Requirements Specification
REST	Representational State Transfer
API	Application Programming Interface
JWT	JSON Web Token (RFC 7519)
RBAC	Role-Based Access Control
SPA	Single Page Application
SQL	Structured Query Language
ACID	Atomicity, Consistency, Isolation, Durability

UI / UX	User Interface / User Experience
CORS	Cross-Origin Resource Sharing

1.3 Intended Audience and Reading Suggestions

This document is targeted toward multiple categories of readers, each of whom should navigate the contents according to their technical objectives:

- **Project Stakeholders and Technical Evaluators:** Sections 1 (Introduction), 2 (Overall Description), and Appendix (Analysis of Models) provide an executive overview of the system's operational scope, institutional feasibility, and architectural rigor.
- **Software Developers and Full-Stack Engineers:** Sections 3 (External Interface Requirements), 4 (System Features), and 6.1 (Conceptual Data Model) establish the definitive behavioral contract, input-processing-output criteria, error conditions, and database structures required for code generation and maintenance.
- **Quality Assurance and Test Automation Engineers:** Section 4 (System Features) and Section 5 (Other Nonfunctional Requirements) formulate the verification matrix, boundary conditions, exception rules, and performance thresholds against which unit, integration, and end-to-end test suites are validated.
- **Institutional Administrators and Facility Wardens:** Section 2.2 (Product Functions), Section 2.3 (User Classes), and Section 5.5 (Business Rules) outline operational policies, permissions models, and institutional workflows enforced by the platform.

1.4 Product Scope

The Hostel & Mess Management System (HMMS) is an integrated enterprise-grade management portal engineered to digitize, unify, and automate residential campus governance in higher education institutions. Historically, university hostels have relied on fragmented paper registers, physical ledgers, verbal notifications, and manual sign-in sheets. This legacy approach is prone to clerical errors, unauthorized out-passes, unaccounted meal waste, delayed maintenance responses, and financial reconciliation discrepancies.

HMMS delivers a unified, centralized web platform engineered on modern client-server principles. The system orchestrates four key administrative domains:

- **1. Student Residential Life:** Enabling self-service visibility into room assignment, meal attendance metrics, semester dues, digital out-pass leave requests, and maintenance complaint tickets.
- **2. Warden Residential Supervision:** Providing automated physical occupancy rosters, real-time bed allocation toggles, digital leave request evaluation, room inspection logging, and roll-call records.

- **3. Mess and Dining Operations:** Managing daily dining menus across four meal tiers (Breakfast, Lunch, Snacks, Dinner), recording student dining check-ins, monitoring turnout, and auditing resident meal fee collections.
- **4. System Administration:** Centralizing role-based provisioning of wardens, mess managers, and student residents, maintaining hostel building and room master inventories, and generating institutional audit reports.

By deploying HMMS, the institution eliminates manual paperwork, enforces strict campus security and accountability, ensures equitable hostel resource allocation, eliminates food waste through meal demand predictability, and maintains transparent financial audit trails.

1.5 References

- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications, IEEE Computer Society, 1998.
- Fielding, R. T., Architectural Styles and the Design of Network-based Software Architectures, Doctoral Dissertation, University of California, Irvine, 2000.
- Internet Engineering Task Force (IETF) RFC 7519, JSON Web Token (JWT), May 2015.
- World Wide Web Consortium (W3C), Web Content Accessibility Guidelines (WCAG) 2.1, June 2018.
- Elmasri, R., and Navathe, S. B., Fundamentals of Database Systems, 7th Edition, Pearson, 2016.
- React Documentation, React: The Library for Web and Native User Interfaces, Meta Platforms, Inc., 2024.
- Node.js Foundation, Node.js Technical Documentation and Architecture, OpenJS Foundation, 2024.

2. Overall Description

2.1 Product Perspective

HMMS is an autonomous, self-contained, web-based software application engineered to replace manual hostel record books and ad-hoc spreadsheets. While operating as a complete standalone system, HMMS is structurally designed following a layered, decoupled Client-Server architecture that integrates with existing institutional local area networks (LAN) or public cloud infrastructure.

The three primary architectural tiers include:

- **Client Tier:** React 18 Single Page Application (Vite Build Engine + Tailwind CSS) providing responsive interfaces across desktop, tablet, and mobile devices with Axios HTTP interceptor pipelines for token handling.

- **Application Tier:** Node.js and Express.js asynchronous runtime environment providing stateless REST API routers, role-based access control middleware, input validation, and business logic controllers.
- **Data Tier:** MySQL 8.0 Enterprise Relational Database Management System providing connection pooling, ACID transaction semantics, referential integrity via foreign key constraints, and 3NF normalized schemas.

2.2 Product Functions

The primary functions executed by the HMMS software ecosystem are categorized across six principal operational pillars:

- **1. User Identity & Access Management:** Secure cryptographically salted credential authentication via JSON Web Tokens (JWT) partitioned across four distinct authorization levels with client-side route guarding.
- **2. Residential Inventory & Bed Allocation:** Building, wing, and room master catalogue tracking room types (Single, Double, Triple), operational status, maximum bed capacities, and real-time warden allocation assignment.
- **3. Multi-Session Campus & Dining Attendance:** Three-tier meal session recording (Breakfast, Lunch, Dinner) as well as daily hostel night roll-call tracking with interactive check-in rosters supporting bulk toggles and 75% residency compliance calculation.
- **4. Residential Leave & Out-Pass Governance:** Self-service digital out-pass filing for overnight campus departures with destination, emergency contact, and duration timestamps, paired with supervisory Warden approval/rejection queues.
- **5. Institutional Billing & Arrears Audit:** Automated ledger creation calculating assessed fees, recorded student settlements, and outstanding balances across room occupancy, mess subscriptions, and campus maintenance dues.
- **6. Campus Maintenance & Grievance Rectification:** Student issue reporting categorized under structural room maintenance, electrical fixtures, plumbing, mess quality, or general campus inquiries with administrative resolution logging.

2.3 User Classes and Characteristics

The platform serves four primary user classes with distinct operational responsibilities, educational backgrounds, and system interaction profiles:

User Class	Technical Expertise	Usage Frequency	Primary System Objectives
Student Resident	Moderate to High	Daily (multiple times)	Review meal menus, monitor attendance compliance, check room allocation, submit leave out-passes, verify payment dues, log

			facility maintenance tickets.
Hostel Warden	Moderate	Daily (work shifts)	Supervise wing occupancy, execute bed allocations, review and approve/reject leave applications, conduct night roll-calls, investigate complaints, audit fee ledgers.
Mess Manager	Basic to Moderate	Daily (meal schedules)	Manage published daily meal menus, execute resident check-ins for breakfast, lunch, and dinner, monitor dietary preferences, audit dining fee reports.
System Administrator	High	Weekly to Monthly	Provision supervisor accounts (Wardens, Mess Managers), manage campus room master inventories, oversee student resident admissions, ensure database integrity.

2.4 Operating Environment

Client-Side Environment

- **Browser Compatibility:** Google Chrome 90+, Mozilla Firefox 88+, Apple Safari 14+, Microsoft Edge 90+, Opera 76+.
- **Device Support:** Responsive layout support across Smartphones (min width 360px), Tablets (768px - 1024px), Laptops (1280px - 1440px), and Ultra-wide Desktop Monitors (1920px+).
- **Client Execution Engine:** ECMAScript 2020 (ES11) compliant JavaScript runtime.

Server-Side Environment

- **Runtime Environment:** Node.js version 18.x LTS or 20.x LTS.
- **Operating System:** Platform independent; verified on Ubuntu Linux 22.04 LTS, Debian 12, Red Hat Enterprise Linux 9, Windows Server 2022, and Windows 10/11 Professional.
- **Web Server / Reverse Proxy:** Express.js native HTTP server, optionally fronted by NGINX or Apache HTTP Server for SSL termination and static asset caching.

Database Environment

- **Database Engine:** MySQL Enterprise Server 8.0 or MariaDB 10.6+.
- **Storage Engine:** InnoDB supporting ACID transactions, row-level locking, and foreign key constraints.
- **Character Encoding:** utf8mb4 with utf8mb4_unicode_ci collation supporting internationalized names and full Unicode glyph sets.

2.5 Design and Implementation Constraints

- **1. Architectural Decoupling:** The frontend SPA and backend REST API must communicate strictly through stateless JSON payloads over HTTP/HTTPS. No serverside HTML rendering or session state storage is permitted in the Express runtime.
- **2. Stateless Authentication:** User identity must be authenticated via digitally signed JSON Web Tokens (JWT) adhering to RFC 7519. Secret keys must be managed through secure environment variables (.env) and never checked into source control.
- **3. Database Normalization:** All relational schemas must adhere to Third Normal Form (3NF) to eliminate data redundancy, anomalous deletions, and update discrepancies.
- **4. CSS and UI Framework Constraints:** Frontend user interfaces must utilize pure Tailwind CSS utility classes and modern vanilla CSS tokens. Bulky third-party UI component suites that inflate bundle sizes or introduce rigid legacy styles are strictly avoided.
- **5. Regulatory & Privacy Compliance:** Student personal records, guardian contact telephone numbers, and financial payment histories must be protected with access controls preventing cross-student data leakage.

2.6 User Documentation

HMMS includes a comprehensive documentation suite delivered alongside the system distribution:

- **System Administrator Installation & Setup Manual:** Technical instructions detailing MySQL database creation, .env parameter configuration, npm package provisioning, database migration scripts, and production process management via PM2.
- **Warden & Mess Manager Operational Guide:** Illustrated task-oriented walkthroughs covering room allocation, multi-session meal check-in, out-pass verification, and billing report extraction.
- **Student Resident Digital Portal Guide:** Accessible online portal documentation explaining leave filing timelines, minimum attendance thresholds, grievance submission procedures, and account security.
- **Interactive REST API Documentation:** Formal OpenAPI/Swagger specification detailing all JSON payload schemas, authorization headers, HTTP response status codes, and error models.

2.7 Assumptions and Dependencies

- **Campus Network Connectivity:** It is assumed that students, wardens, and mess supervisors have access to campus Wi-Fi, LAN, or cellular mobile networks to access the web application.
- **Device Availability:** It is assumed that wardens and mess managers possess desktop computers, laptops, or tablets capable of rendering modern HTML5/CSS3 web browsers.

- **Institutional Master Data:** It is assumed that initial student academic admissions records (Roll Numbers, Departments, Assigned Academic Years) are provisioned through verified institutional enrolment records.
- **Database Persistence:** The system depends on a continuous, uninterrupted connection to the MySQL database engine. Database clustering or localized automatic restarts must maintain high availability.

3. External Interface Requirements

3.1 User Interfaces

The HMMS user interface is engineered according to commercial SaaS product design standards, prioritizing visual ergonomics, disciplined spacing, and accessibility.

Visual Design System

- **Color Palette:** Deep Obsidian Blue (#060C1B) for application canvas, Slate Midnight (#0F172A) for navigation sidebars, Card Charcoal (#111827) for cards, Elevated Navy (#1A2338) for secondary surfaces, Royal Blue (#2563EB) for brand accents, Success Emerald (#16A34A), Warning Amber (#D97706), and Danger Crimson (#DC2626) for semantic badges.
- **Typographic Scale:** Inter typography font family applied globally; Page Titles at 42px Bold, Section Titles at 24px SemiBold, Statistical Metric Values at 32px Bold, Table Headers & Body at 15px Medium/Regular, and Secondary Badges at 13px Regular.
- **Spatial System:** Rigid 8px dimensional grid enforcing 24px card paddings (p-6), 32px major section gaps (space-y-8), 16px grid card gutters (gap-4/gap-6), and 40–44px uniform button heights.

Layout Blueprints

Every role-based portal implements a persistent, fixed-width sidebar (w-64) with rounded geometry, soft elevation shadows, aligned Lucide icons, and a filled blue active pill indicator. The top header provides persistent breadcrumbs, user identification avatars with initial-based fallbacks, and notification triggers. Tables feature sticky column headers, rounded outer containers, and soft row hover transitions.

3.2 Hardware Interfaces

- **Client Processing Hardware:** Standard personal computing devices equipped with minimum 2 GB RAM, dual-core processors, and display monitors with minimum resolution of 360 x 640 pixels (mobile) or 1366 x 768 pixels (workstation).
- **Server Infrastructure:** Host computing server equipped with minimum 2 vCPUs, 4 GB RAM, and 20 GB solid-state disk (SSD) storage running standard server operating systems.

3.3 Software Interfaces

- **Operating System Interfaces:** Node.js system runtime accessing filesystem directories for log output, environment configuration files (.env), and process signal monitoring (SIGINT, SIGTERM).
- **Database Engine Management:** Communicates with MySQL 8.0 via the mysql2/promise driver over standard TCP/IP socket connections utilizing connection pooling (maximum 10 concurrent connections per node process).
- **HTTP Client Communication:** Frontend Axios client library configured with dynamic environment-driven base URLs and asynchronous request/response interceptors that inject Bearer JWT authorization headers into outgoing requests.
- **Component Iconography:** Lucide React modern vector icon package for consistent, responsive SVG vector graphics rendering across all viewports.

3.4 Communications Interfaces

- **Transport Layer Protocol:** Hypertext Transfer Protocol Secure (HTTPS) over Transport Layer Security (TLS 1.2 or TLS 1.3) for all production traffic; HTTP/1.1 permitted during local development.
- **Data Serialization:** JavaScript Object Notation (JSON) format adhering to MIME type application/json for all API request bodies and response payloads.
- **Cross-Origin Resource Sharing (CORS):** Backend Express CORS middleware configured to validate and permit trusted origins (including localhost, institutional domain, and cloud hosting URLs) with credentials enabled, preventing cross-site scripting and unauthorized domain access.

4. System Features

4.1 User Authentication and Access Control

4.1.1 Description and Priority

HMMS mandates secure credential authentication for all system actors. The system implements stateless, role-partitioned authentication utilizing JSON Web Tokens.
Priority: High (Core Security Foundation).

4.1.2 Stimulus/Response Sequences

- **User Action:** The user navigates to /login, selects their operational portal tab (Student, Warden, Mess Manager, or Admin), enters their registered email address and password, and clicks 'Authenticate Portal'.
- **System Response:** The clientside application validates input completeness and dispatches an HTTP POST request to /api/auth/login.

- **Backend Processing:** The server validates the existence of the email, verifies the password hash, asserts that the user's role matches the requested portal, generates an encrypted JWT token, and returns user identity metadata.
- **Client Ingestion:** The client stores the JWT in persistent storage, updates the global AuthContext state, and navigates the user to their designated dashboard.

4.1.3 Functional Requirements

- **REQ-AUTH-01:** The system shall authenticate users against the USER relational table via verified email address and cryptographic password match.
- **REQ-AUTH-02:** Upon successful verification, the system shall generate a cryptographically signed JSON Web Token containing user_id, email, and role, signed with an HMAC-SHA256 secret key, configured with an expiration period of 24 hours.
- **REQ-AUTH-03:** The system shall reject login attempts where the authenticated user role does not match the active login portal selection.
- **REQ-AUTH-04:** The frontend application shall intercept all protected route transitions and redirect unauthenticated or unauthorized requests to /login.
- **REQ-AUTH-05:** The frontend Axios interceptor shall automatically attach the authorization header 'Authorization: Bearer <token>' to all HTTP requests directed to /api/*.

4.2 Student Resident Onboarding and Profiling

4.2.1 Description and Priority

Allows Administrators to register new student hostel residents, provision access credentials, and maintain academic directory profiles.
Priority: High.

4.2.2 Stimulus/Response Sequences

- **User Action:** Admin opens the Student Roster portal (/admin/students), expands the 'Enrol New Student' form, fills in student demographic details (Name, Email, Password, Roll Number, Phone, Department, Year of Study, Guardian Contact), and submits.
- **System Response:** Form inputs are verified; POST request dispatches to /api/admin/students.
- **Backend Processing:** System executes an atomic transaction creating the base USER record with role 'STUDENT', retrieves the inserted user_id, and creates the associated STUDENT record with roll number and academic parameters.
- **System Response:** System alerts success and triggers an automatic roster re-fetch.

4.2.3 Functional Requirements

- **REQ-STU-01:** The system shall enforce uniqueness on student roll numbers and institutional email addresses.
- **REQ-STU-02:** The system shall store academic details including Department and Year of Study (1 through 4).

- **REQ-STU-03:** The system shall record student contact phone numbers and mandatory emergency guardian contact numbers.
- **REQ-STU-04:** Deletion of a student record shall cascade appropriately, clearing active room allocations while archiving past financial transaction records.

4.3 Room Inventory and Occupancy Allocation

4.3.1 Description and Priority

Enables the administration to provision physical hostel rooms, define occupancy capacities, track vacancies, and empowers Wardens to allocate residents to beds.
Priority: High.

4.3.2 Stimulus/Response Sequences

- **Room Creation:** Admin inputs Room Number, Room Tier (Single, Double, Triple), and Capacity (1 to 6) on /admin/rooms. System inserts record into ROOM with initial occupied_count = 0 and status AVAILABLE.
- **Room Allocation:** Warden views active room cards on /warden/rooms. The card displays occupancy progress (e.g., '1 of 2 Beds Filled'). The Warden selects an unallocated resident from the dropdown and clicks 'Allocate Student'.
- **Processing:** System verifies that occupied_count < capacity. It updates the student's room_id in the database, increments the room's occupied_count, and if occupied_count == capacity, sets room status to FULL.
- **Student Feedback:** When the student resident logs into their portal (/student/room), the system renders their assigned room number, block wing, room type, and occupancy status.

4.3.3 Functional Requirements

- **REQ-ROOM-01:** The system shall track room inventory with unique room identifiers, room numbers, types (Single, Double, Triple), capacities, and statuses (Available, Full).
- **REQ-ROOM-02:** The system shall prohibit the allocation of any student to a room whose occupied_count is equal to or exceeds its designated capacity.
- **REQ-ROOM-03:** The system shall automatically compute room occupancy percentage and visually display dynamic gauge indicators.
- **REQ-ROOM-04:** Wardens shall have manual toggle overrides to mark a room as Available or Full during maintenance closures.

4.4 Multi-Session Daily Attendance Tracking

4.4.1 Description and Priority

Provides granular tracking of resident presence across three daily dining sessions (Breakfast, Lunch, Dinner) as well as daily hostel residency roll-calls.
Priority: High.

4.4.2 Stimulus/Response Sequences

- **Warden / Manager Action:** The supervisor navigates to the attendance portal (/warden/attendance or /mess-manager/attendance).
- **Roster Display:** The system renders the registered student resident roster. Each student card displays meal session status indicators.
- **Toggle Event:** The supervisor clicks the 'Present' or 'Absent' segmented toggle button for a resident.
- **System Response:** System performs an optimistic UI update, changes the badge to active green or neutral crimson, and dispatches a background POST request to /api/attendance with student_id, date, meal_type, and present (boolean).
- **Database Processing:** The server performs an INSERT ... ON DUPLICATE KEY UPDATE query, updating existing logs or inserting new entries.
- **Student Calculation:** The student attendance view (/student/attendance) recalculates overall attendance percentage: (Total Present Sessions / Total Evaluated Sessions) * 100, flagging compliance against the 75% benchmark.

4.4.3 Functional Requirements

- **REQ-ATTN-01:** The system shall support attendance tracking across discrete meal types: BREAKFAST, LUNCH, DINNER, and general DAILY roll-call.
- **REQ-ATTN-02:** Attendance records shall be uniquely constrained by the composite key (student_id, date, meal_type).
- **REQ-ATTN-03:** The system shall maintain optimistic UI updates allowing supervisors to record rapid check-ins during high-volume dining hall entry without interface freezing.
- **REQ-ATTN-04:** The student dashboard shall compute overall percentage turnout and display warning indicators if attendance drops below 75%.

4.5 Digital Leave Application and Warden Approval

4.5.1 Description and Priority

Manages student requests for overnight campus departures, weekend out-passes, and vacation leaves, replacing paper leave slips with an audited digital workflow.
Priority: High.

4.5.2 Stimulus/Response Sequences

- **Student Application:** Student accesses /student/leave, enters departure date (from_date), return date (to_date), and comprehensive reason/destination, and submits.
- **Backend Submission:** System validates that to_date >= from_date, inserts record into LEAVE_APPLICATION with status PENDING, and sets applied_date = CURRENT_TIMESTAMP.

- **Warden Review:** Warden opens /warden/leave-requests. The pending request queue highlights the student's name, roll number, requested date range, and reason.
- **Decision Execution:** Warden clicks 'Approve' or 'Reject'. System dispatches PUT request to /api/warden/leaves/:id updating status to APPROVED or REJECTED.
- **Real-time Status Update:** Student application table updates immediately to display an approved green pill badge or rejected crimson badge.

4.5.3 Functional Requirements

- **REQ-LEAVE-01:** The system shall validate that leave return dates are chronologically equal to or subsequent to the departure date.
- **REQ-LEAVE-02:** All submitted leave applications shall default to the PENDING review state.
- **REQ-LEAVE-03:** Only authenticated users with the Warden role shall have authorization to transition leave application states to APPROVED or REJECTED.
- **REQ-LEAVE-04:** The system shall preserve a permanent chronological audit log of all historical leave decisions.

4.6 Fee Ledger, Billing Reports, and Arrears Audit

4.6.1 Description and Priority

Audits hostel room fees, mess catering dues, and miscellaneous charges, tracking payment settlements and outstanding student arrears.
Priority: High.

4.6.2 Stimulus/Response Sequences

- **Ledger Audit:** Warden or Mess Manager opens /fees/report.
- **Aggregated Processing:** Server queries the FEE table grouped by student, calculating SUM(amount) as Total Assessed, SUM(amount WHERE status = 'PAID') as Amount Cleared, and the difference as Outstanding Arrears.
- **Interface Render:** The report renders KPI metric cards: Total Assessed Revenue, Total Collected, Total Outstanding, and percentage Collection Rate.
- **Filtering:** Supervisors filter between 'All Records', 'Cleared Accounts', and 'Pending Dues', or execute real-time searches by roll number or student name.

4.6.3 Functional Requirements

- **REQ-FEE-01:** The system shall track individual fee line items with amount, fee category, due date, and payment status (PAID, PENDING).
- **REQ-FEE-02:** The system shall automatically compute student balance arrears and identify students with zero dues versus outstanding debt.
- **REQ-FEE-03:** Fee reports shall provide instant search filtering across roll numbers, student names, and payment status categories.

4.7 Facility Maintenance and Grievance Ticketing

4.7.1 Description and Priority

Allows residents to report room fixture defects, electrical failures, plumbing problems, or food hygiene grievances, and empowers Wardens to log inspections and resolutions.
Priority: Medium.

4.7.2 Stimulus/Response Sequences

- **Ticket Logging:** Student accesses /student/complaints, selects category (ROOM, MAINTENANCE, MESS, OTHER), inputs problem description, and submits.
- **Backend Processing:** System normalizes category input against database ENUM values, inserts record into COMPLAINT with status OPEN.
- **Warden Resolution:** Warden views /warden/complaints, filters by OPEN or RESOLVED, clicks 'Mark Resolved', inputs administrative resolution notes, and saves.
- **Resolution Recording:** System updates ticket with status = 'RESOLVED', resolved_date = NOW(), and stores resolution notes for student review.

4.7.3 Functional Requirements

- **REQ-COMP-01:** The system shall accept grievance tickets categorized under ROOM, MAINTENANCE, MESS, or OTHER.
- **REQ-COMP-02:** All newly submitted tickets shall initialize with status = 'OPEN'.
- **REQ-COMP-03:** Wardens shall be required to input resolution notes when marking a complaint as resolved.

4.8 Mess Dining Menu and Catering Operations

4.8.1 Description and Priority

Displays the official institutional meal timetable across breakfast, lunch, evening snacks, and dinner, specifying timings and dietary categories.
Priority: Medium.

4.8.2 Functional Requirements

- **REQ-MESS-01:** The system shall maintain scheduled meal menu timetables across four standardized daily dining sessions: Breakfast, Lunch, Evening Snacks, and Dinner.
- **REQ-MESS-02:** The system shall display specific serving time windows and dietary classifications (e.g., South Indian Special, Full Thali Meal).
- **REQ-MESS-03:** The student mess portal shall provide institutional food safety guidelines, catering certifications, and special sick diet request protocols.

4.9 Supervisory Staff Provisioning and Administrative Governance

4.9.1 Description and Priority

Enables System Administrators to appoint, configure, and decommission supervisory personnel (Wardens and Mess Managers).
Priority: High.

4.9.2 Functional Requirements

- **REQ-ADM-01:** The system shall allow Administrators to create new user accounts assigned with WARDEN or MESS_MANAGER roles.
- **REQ-ADM-02:** The system shall provide deletion capabilities for decommissioned staff accounts with referential integrity enforcement.
- **REQ-ADM-03:** The Admin dashboard shall display aggregate institution-wide KPI statistics: Total Enrolled Students, Active Wardens, Mess Managers, and Hostel Room Inventory.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- **Response Latency:** The REST API backend shall process and respond to 95% of standard read queries within 250 milliseconds under standard campus network loads.
- **Client Render Speed:** Initial Single Page Application bundle load and first contentful paint (FCP) shall complete within 1.2 seconds on standard broadband connections.
- **Concurrent Load Capacity:** The system architecture shall support a minimum of 500 concurrent active user sessions without service degradation or database thread exhaustion.
- **Optimistic State Updates:** Interactive roll-call and meal check-in buttons shall reflect state changes within 50 milliseconds on the client interface to support rapid dining hall queues.

5.2 Safety Requirements

- **Data Loss Prevention:** All database write operations modifying student residential status, bed allocations, or payment receipts must be committed within transactional boundaries (ACID) to prevent partial state corruption during hardware failure.
- **Emergency Contact Integrity:** Guardian contact phone numbers stored in the STUDENT table must be strictly preserved without truncation to ensure immediate availability during campus medical emergencies.

5.3 Security Requirements

- **Credential Storage:** User passwords must never be stored in plaintext. Passwords must be hashed using strong cryptographic hashing algorithms (bcrypt with a minimum work factor of 10).

- **Cryptographic Token Verification:** All protected API endpoints must validate incoming JWT tokens against signature corruption, tampering, and expiration.
- **Role-Based Authorization:** Backend routes must enforce strict role verification middleware: /api/admin/* restricted to ADMIN, /api/warden/* restricted to WARDEN, /api/mess-manager/* restricted to MESS_MANAGER, and /api/student/* restricted to STUDENT.
- **SQL Injection Prevention:** All SQL queries interacting with the MySQL database must utilize prepared statements with parameterized queries (? placeholders). No string concatenation of user inputs into SQL strings is permitted.
- **Cross-Site Scripting (XSS) Sanitization:** React's virtual DOM natively escapes HTML entity inputs, preventing execution of injected script tags in student complaint or leave text fields.

5.4 Software Quality Attributes

- **Availability:** The system shall maintain an operational availability of 99.5% during academic semesters, scheduling administrative maintenance during off-peak hours (02:00 AM - 04:00 AM).
- **Maintainability:** Source code must adhere to clean modular separation: frontend reusable UI tokens isolated in src/components/ui/, backend routers decoupled from controller business logic, and database configs centralized in config/db.js.
- **Usability:** The UI must adhere to modern SaaS visual guidelines, utilizing high-contrast typography, clear status pill badges, and accessible form labels compliant with WCAG 2.1 Level AA standards.
- **Portability:** The entire application stack must execute consistently across standard operating systems (Windows, Linux, macOS) without modification to underlying application source code.

5.5 Business Rules

- **Rule 1 (Attendance Threshold):** Institutional regulations mandate that student residents maintain an aggregate attendance turnout of at least 75% across academic terms to retain hostel accommodation eligibility.
- **Rule 2 (Single Room Allocation):** A student resident may be allocated to at most one hostel room at any given point in time.
- **Rule 3 (Bed Capacity Cap):** The total number of active student residents allocated to a room shall never exceed the defined physical bed capacity of that room.
- **Rule 4 (Chronological Leave Integrity):** A leave out-pass return date must be chronologically equal to or greater than the departure date.
- **Rule 5 (Warden Authorization):** Only appointed Wardens are legally authorized to review, approve, or reject student residential leave applications.

6. Other Requirements

6.1 Conceptual Data Model and Relational Schemas

The HMMS database schema is normalized to Third Normal Form (3NF) to guarantee transactional integrity and eliminate update anomalies.

Relational Entity Descriptions

- **1. USER:** Represents all system actors. Stores user_id (INT, PK), username (VARCHAR(100)), email (VARCHAR(100), UQ), password (VARCHAR(255)), role (ENUM: ADMIN, WARDEN, MESS_MANAGER, STUDENT), phone (VARCHAR(20)), created_at (TIMESTAMP).
- **2. STUDENT:** Specialized entity for residents. Stores student_id (INT, PK), user_id (INT, FK -> USER), roll_no (VARCHAR(50), UQ), department (VARCHAR(100)), year_of_study (INT), room_id (INT, FK -> ROOM), guardian_contact (VARCHAR(20)).
- **3. WARDEN:** Supervisory personnel. Stores warden_id (INT, PK), user_id (INT, FK -> USER), hostel_block (VARCHAR(50)).
- **4. MESS_MANAGER:** Dining supervisor. Stores manager_id (INT, PK), user_id (INT, FK -> USER), mess_name (VARCHAR(100)).
- **5. ROOM:** Hostel residential rooms. Stores room_id (INT, PK), room_number (VARCHAR(20), UQ), room_type (ENUM: SINGLE, DOUBLE, TRIPLE), capacity (INT), occupied_count (INT), status (ENUM: AVAILABLE, FULL).
- **6. ATTENDANCE:** Multi-session dining and roll-call. Stores attendance_id (INT, PK), student_id (INT, FK -> STUDENT), date (DATE), meal_type (ENUM: BREAKFAST, LUNCH, DINNER, DAILY), present (BOOLEAN). Composite unique key on (student_id, date, meal_type).
- **7. LEAVE_APPLICATION:** Out-pass departure requests. Stores leave_id (INT, PK), student_id (INT, FK -> STUDENT), from_date (DATE), to_date (DATE), reason (TEXT), status (ENUM: PENDING, APPROVED, REJECTED), applied_date (TIMESTAMP).
- **8. FEE:** Student billing ledgers. Stores fee_id (INT, PK), student_id (INT, FK -> STUDENT), amount (DECIMAL(10,2)), fee_type (VARCHAR(50)), due_date (DATE), status (ENUM: PAID, PENDING, UNPAID), paid_date (TIMESTAMP).
- **9. COMPLAINT:** Maintenance and grievance tickets. Stores complaint_id (INT, PK), student_id (INT, FK -> STUDENT), resolved_by (INT, FK -> WARDEN), category (ENUM: ROOM, MESS, MAINTENANCE, OTHER), description (TEXT), status (ENUM: OPEN, IN_PROGRESS, RESOLVED, REJECTED), submitted_date (TIMESTAMP), resolved_date (TIMESTAMP), resolution_notes (TEXT).

Entity Relationship Mapping Table

Source Entity	Cardinality	Target Entity	Relationship Description	Integrity Rule
USER	1 : 1	**STUDENT**	User credentials map to resident profile	ON DELETE CASCADE
USER	1 : 1	**WARDEN**	User credentials map to warden profile	ON DELETE CASCADE
USER	1 : 1	**MESS_MANAGER**	User credentials map to dining manager	ON DELETE CASCADE
ROOM	1 : N	**STUDENT**	Room houses multiple residents up to capacity	ON DELETE SET NULL
STUDENT	1 : N	**ATTENDANCE**	Student accumulates multi-session roll-calls	ON DELETE CASCADE
STUDENT	1 : N	**LEAVE_APP**	Student files chronological leave out-passes	ON DELETE CASCADE
STUDENT	1 : N	**FEE**	Student is assessed semester fee invoices	ON DELETE CASCADE
STUDENT	1 : N	**COMPLAINT**	Student logs maintenance grievance tickets	ON DELETE CASCADE
WARDEN	1 : N	**COMPLAINT**	Warden inspects and resolves tickets	ON DELETE SET NULL

6.2 Data Backup and Disaster Recovery Strategy

- **1. Automated Logical Backups:** The system utilizes native mysqldump utilities scheduled via server cron jobs to generate daily compressed .sql.gz snapshots of the HMMS database at 03:00 AM UTC.
- **2. Point-in-Time Recovery (PITR):** MySQL binary logging (binlog) is enabled to allow database restoration to any precise second preceding an accidental administrative deletion.
- **3. Restoration Verification:** Quarterly mock restoration tests are executed on staging servers to verify backup integrity and validate recovery time objectives (RTO < 1 hour, RPO < 24 hours).

6.3 Security Considerations and Hardening

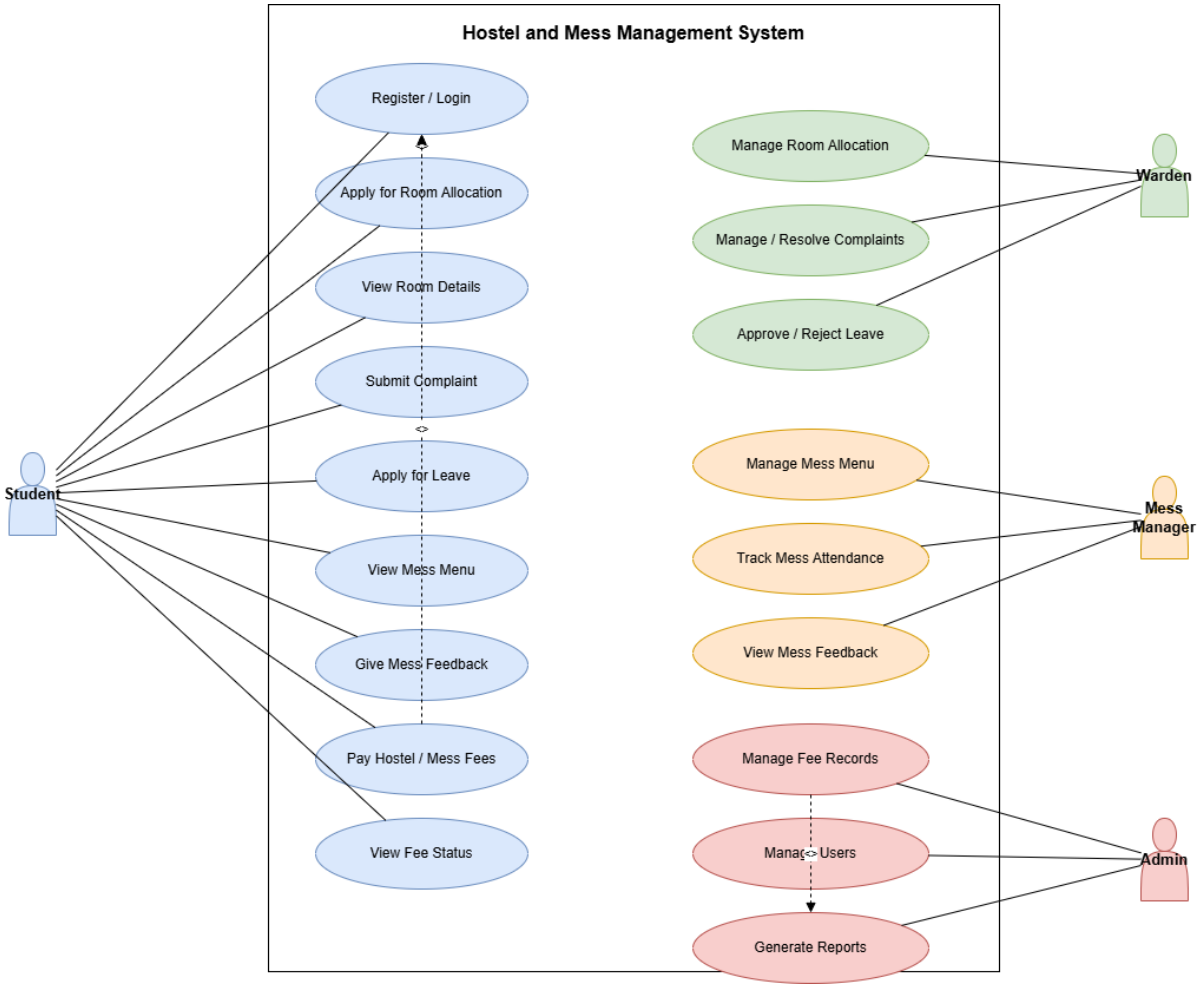
- **Environment Isolation:** Database credentials, JWT secret keys, and server port allocations are strictly isolated within uncommitted .env files.
- **Strict Parameter Normalization:** Backend controllers sanitize all user payloads and validate enumeration values prior to database query execution, eliminating uncaught runtime exceptions.
- **Session Expiration:** Client-side authentication tokens are verified on every view transition; expired or tampered tokens result in immediate memory clearance and redirection to the portal login screen.

6.4 Future System Enhancements

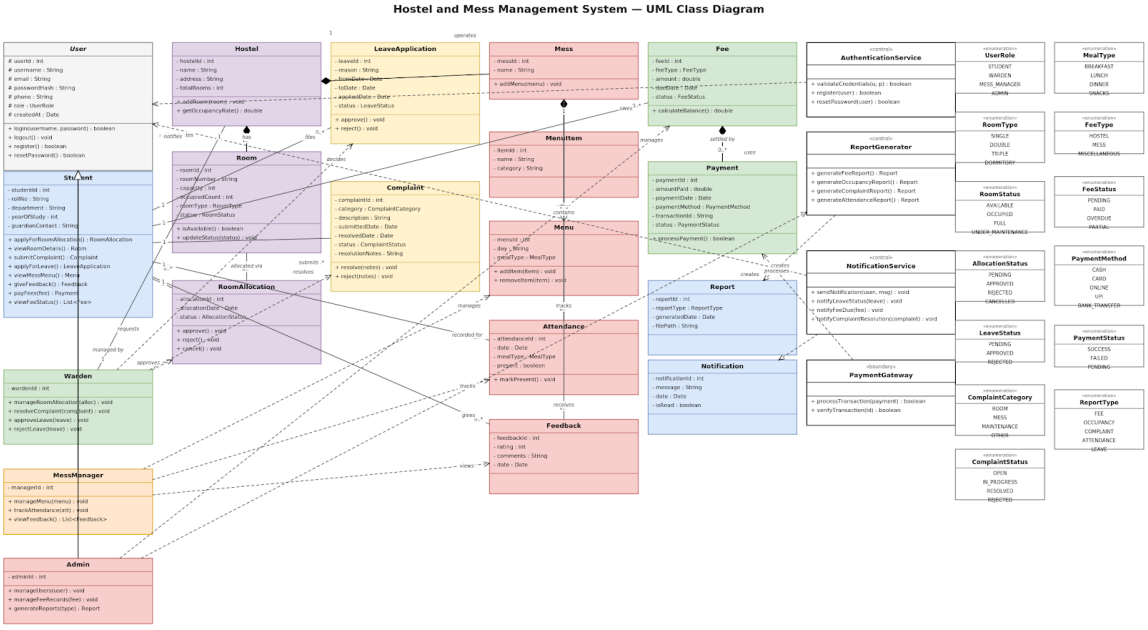
- **Automated Payment Gateway Integration:** Incorporating Razorpay/Stripe webhooks for online student fee settlement via UPI, credit/debit cards, and net banking.
- **Biometric Hardware Integration:** Interfacing physical fingerprint scanners or RFID smart-card turnstiles at dining hall entries with the attendance logging endpoint.
- **SMS / Push Notifications:** Automated SMS alerts dispatched to parents upon Warden approval of overnight student leave out-passes.

Appendix : Analysis Of Models

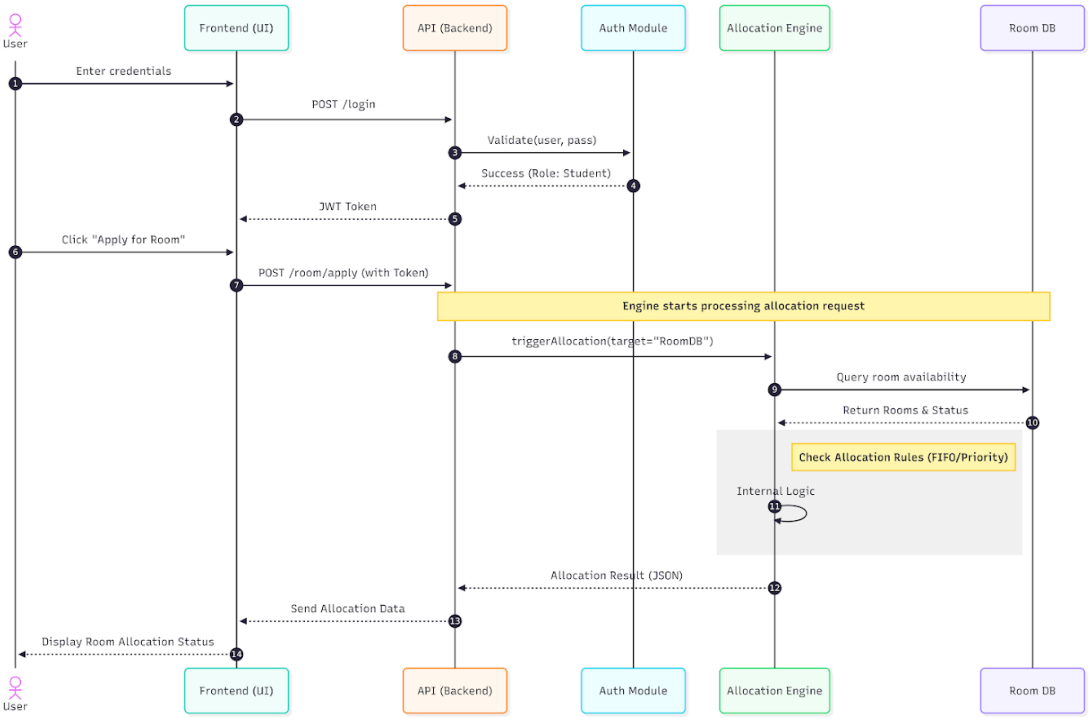
Use Case Diagram



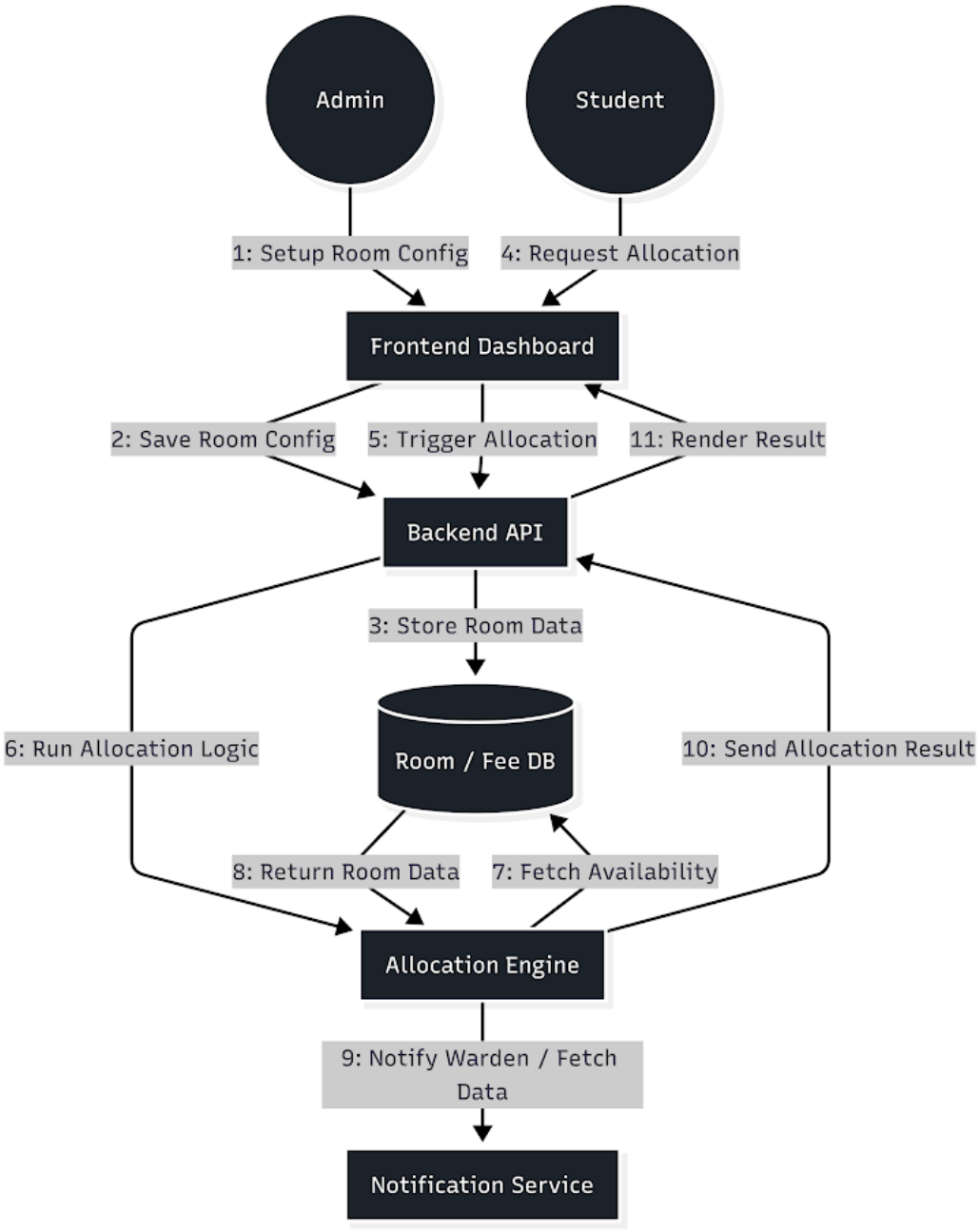
Class Diagram



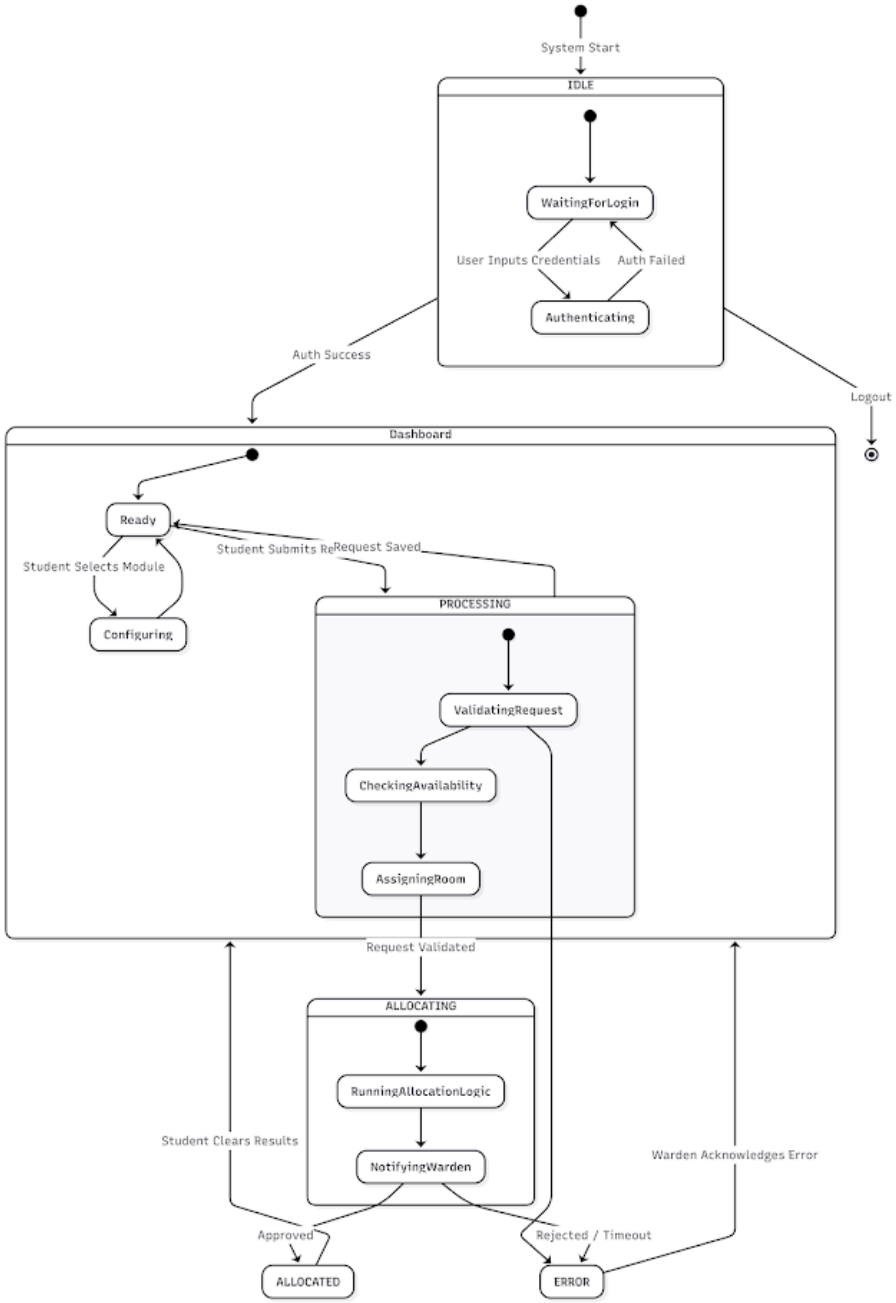
Sequence Diagram



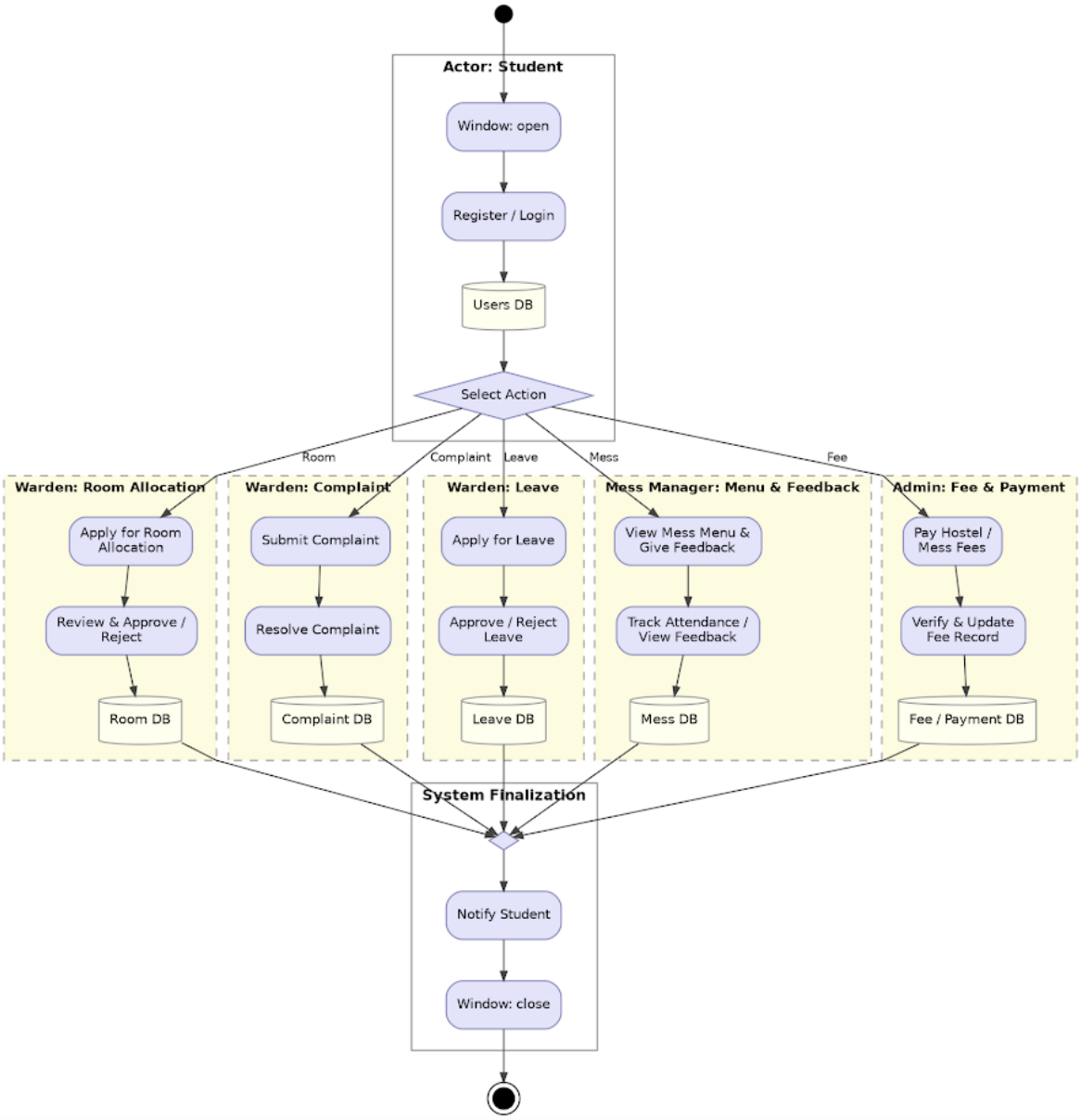
Collaboration Diagram



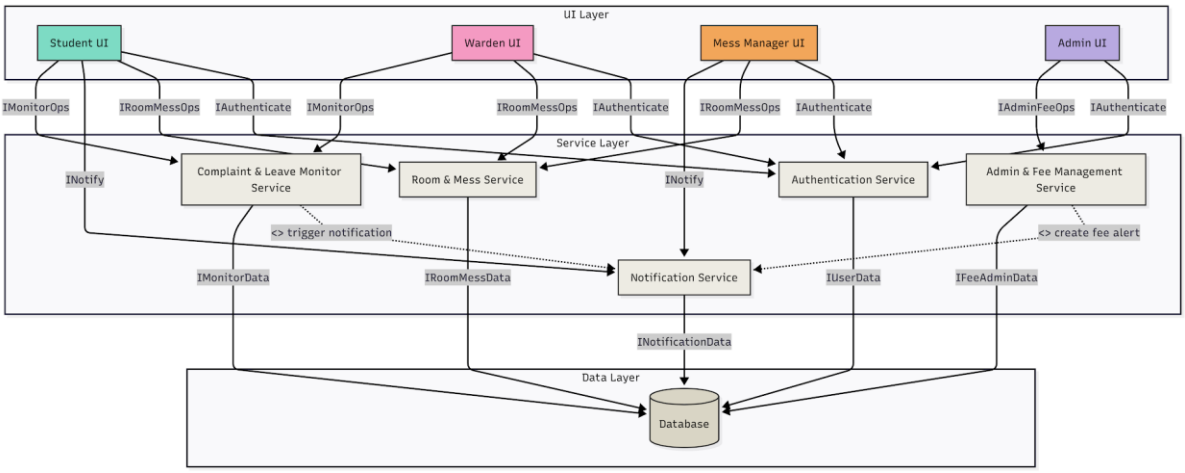
State Chart Diagram



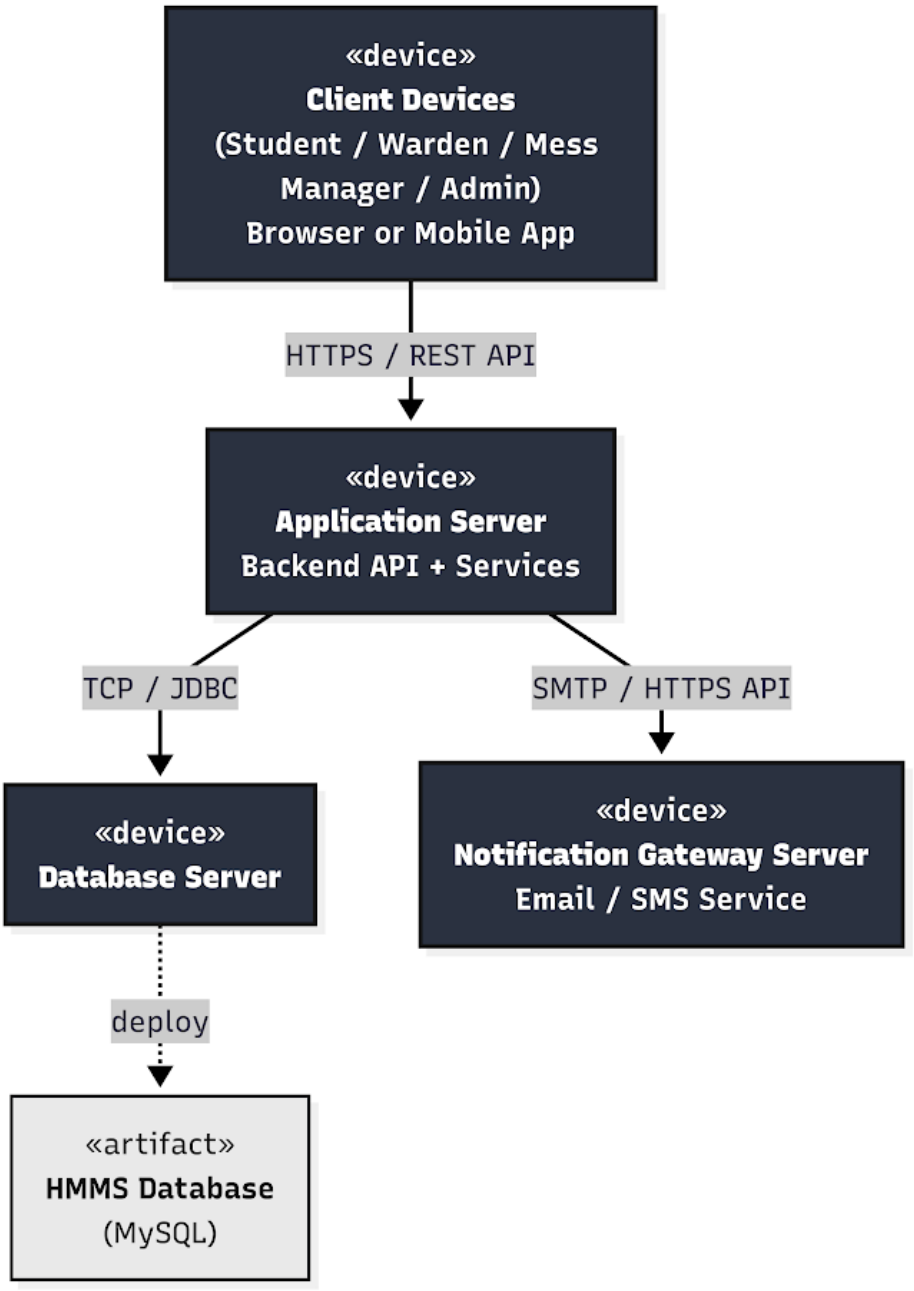
Activity Diagram



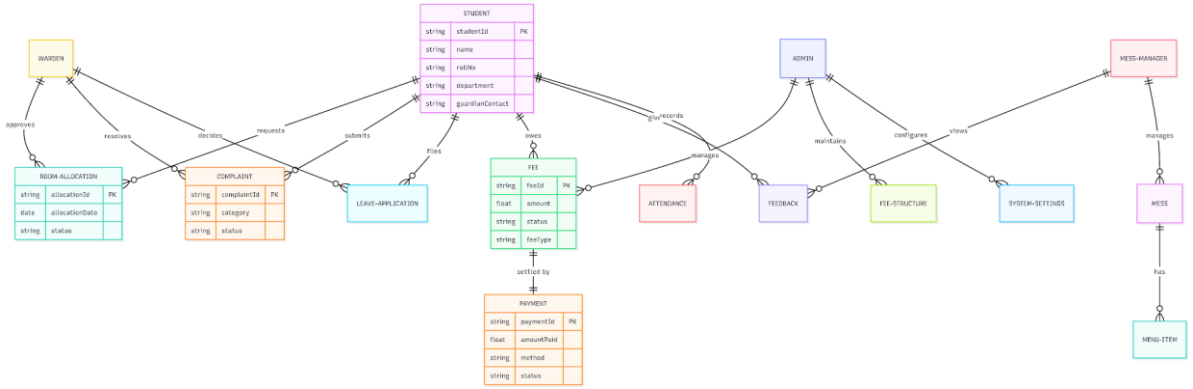
Component Diagram



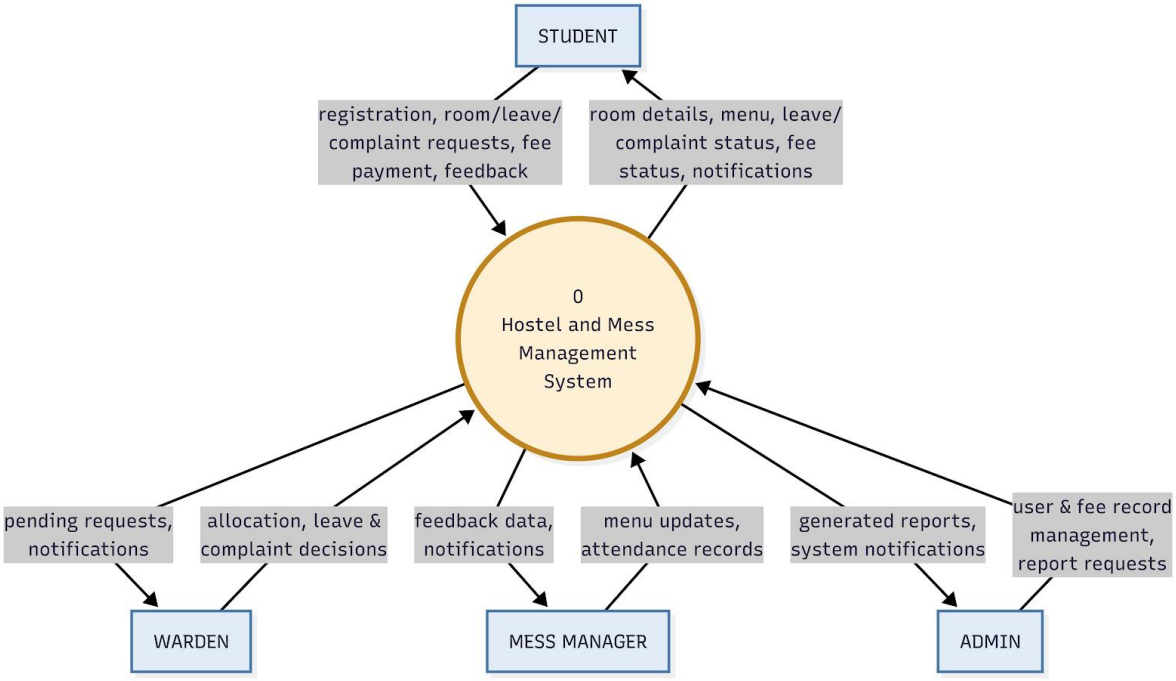
Deployment Diagram



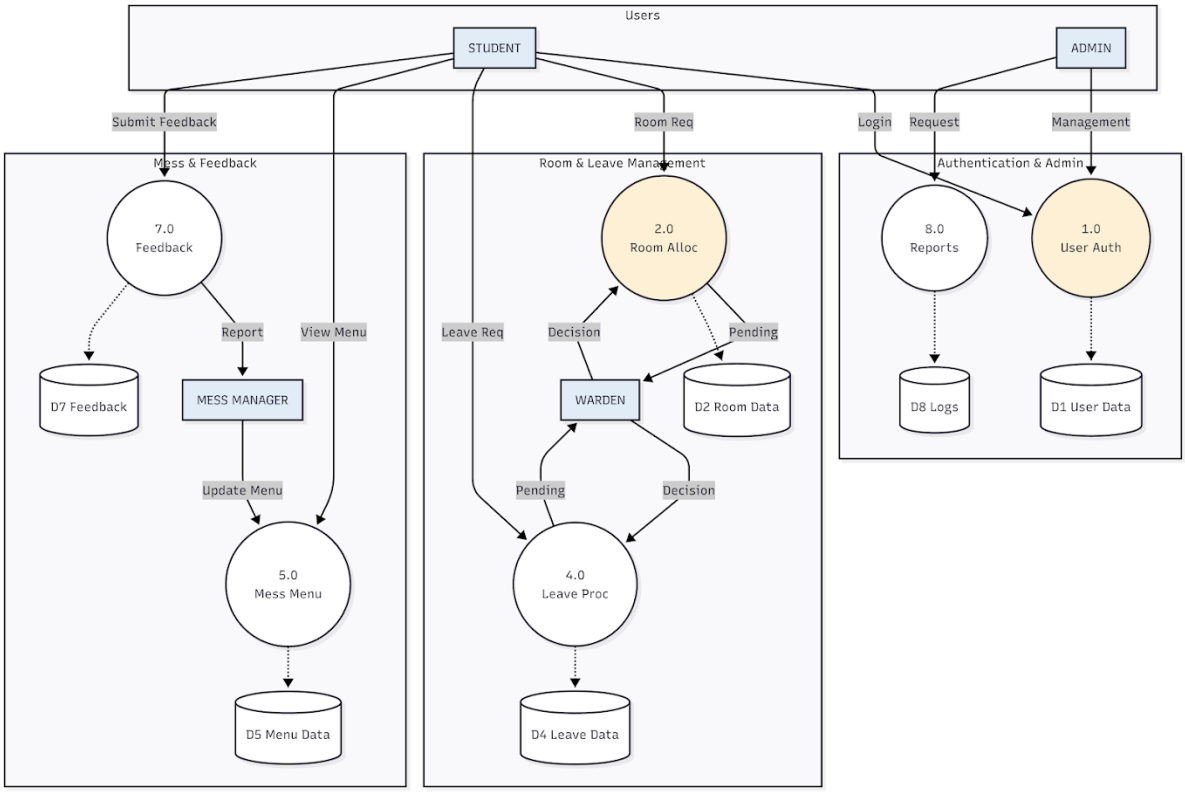
Entity Relationship (ER) Diagram



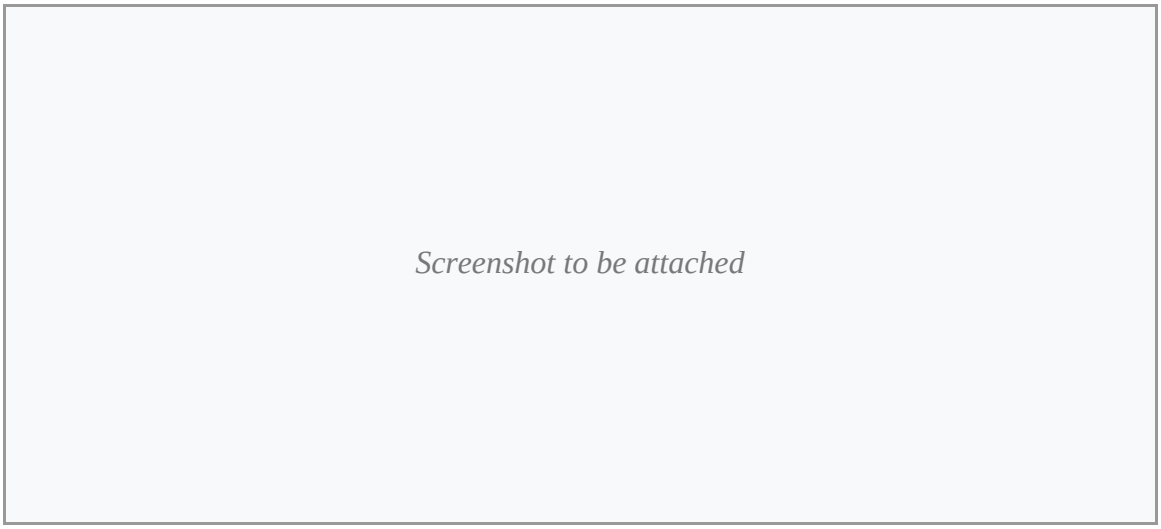
Data Flow Diagram (DFD) - Level 0



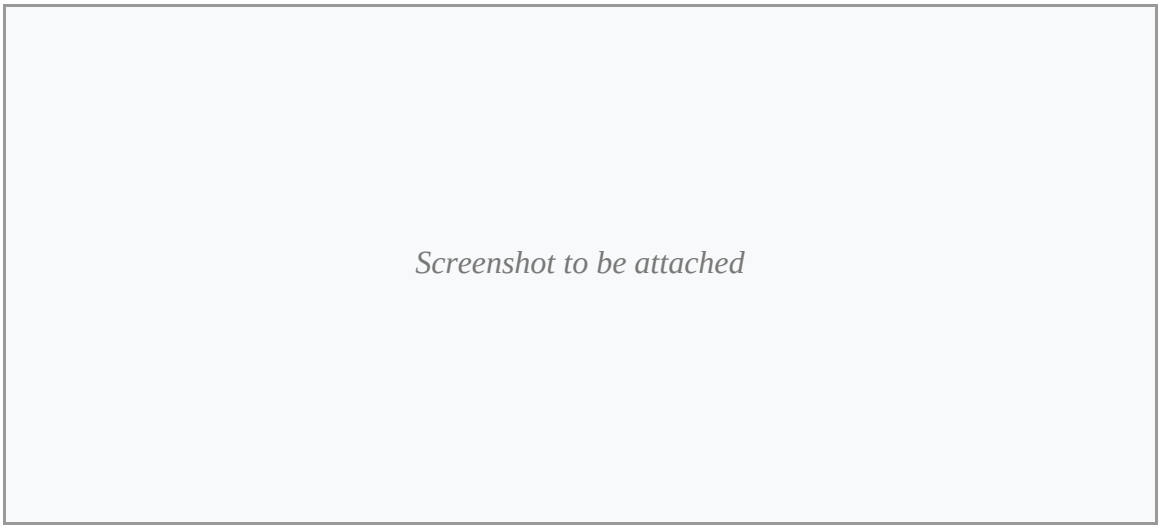
Data Flow Diagram (DFD) - Level 1



Demonstration



Login page



Student Registration & Room Allocation page

Screenshot to be attached

Student Dashboard

Screenshot to be attached

Digital Mess Pass & QR Code Verification

Screenshot to be attached

Mess Cut / Rebate Application & Status

Screenshot to be attached

Fee Ledger, Arrears & Payment Portal

Screenshot to be attached

Attendance & Movement Logging Screen

Screenshot to be attached

Leave & Night-Out Pass Workflow

Screenshot to be attached

Maintenance & Grievance Ticketing Portal

Screenshot to be attached

Warden & Administration Oversight Dashboard

Screenshot to be attached

Mess Dining Menu & Inventory Screen

Screenshot to be attached

Institutional Audit & Room Occupancy Report